Vidtuber

Options Dialog

General

- **Don’t check SSL certificate**

  An SSL certificate is a digital certificate that authenticates a website's identity and enables an encrypted connection. SSL stands for Secure Sockets Layer, a security protocol that creates an encrypted link between a web server and a web browser. If selected this control disables the certificate check.

- **Include the ID in file names**

  Downloaded files have unique identifiers which can be added to the name of each downloaded file.

- **Restrict file names to ASCII characters**

  File names will only have ASCII characters avoiding meta-characters (like &%/\<> etc) and spaces in file names. This option is often recommended to avoid reading problems and possible incompatibilities of your operating system.

- **Embed thumbnail in audio file**

  Embed the thumbnail in the audio file as cover art for some compatible audio formats only. This option includes both writing the thumbnail to the specified path and embedding it in the downloaded audio file.

- **Add metadata to file**

  Embed the metadata to the Audio/Video file. Also embeds chapters/infojson if present.

- **Overwrite all files and metadata**

  When a file with the same name is downloaded more than once to the same destination path it will be completely overwritten and will not take into account the parts/fragments already downloaded previously.

File Preferences

These settings are related to the destination path of the downloaded files. When playlists are downloaded, folder collections will be automatically generated by default.

Download Options

External downloader refers to the back-end used to manage the file download process. yt-dlp uses its own "native" downloader by default, but you can use faster ones if installed on your system.

For example using aria2:

External downloader executable path: **aria2c**

External downloader arguments: **-j 1 -x 1 -s 1**

```
-j, --max-concurrent-downloads=<N>
Set the maximum number of parallel downloads for every queue item. Default: 5


-x, --max-connection-per-server=<NUM>
The maximum number of connections to one server for each download.  Default: 1


-s, --split=<N>
 Download a file using N connections.
```

Another example using aria2:

External downloader executable path: **aria2c**

External downloader arguments: **-x 16 -s 16 -k 1M**

```
-x 16
Set Maximum of 16 connections per server.


-s 16
Splits the file into 16 parts to be downloaded in parallel.


-k 1M
Minimum chunk size (1 Megabyte).
```

See the documentation provided by your external downloader for more details on option arguments.

Network

Using a proxy server

If you are behind a proxy you should set it up correctly using this option. You can use a specific HTTP or HTTPS or SOCKS type proxy. For example, to enable a SOCKS proxy, specify an appropriate scheme like this: **socks5://user:pass@127.0.0.1:1080/.**

Timeouts and Retries

- Timeout

It sets the maximum wait time (in seconds) before the back-end (yt-dlp) considers a connection to be stalled and terminates it due to server inactivity. The default value is 20 seconds, which is the same as the default value for yt-dlp.

How it works:

A. Stall detection: If you are starting a download or are in the middle of a transfer and the server suddenly stops sending data (or fails to respond to the initial request), it will wait for the specified number of seconds.

B. Timeout trigger: Once that time has elapsed without receiving any signal, the connection is considered “dead.”

C. Integration with retries: When a timeout occurs, it does not give up immediately; instead, it closes the stalled connection and opens a new one, decrementing the count of available attempts (as configured via **Retries** or **Fragment Retries**).

- Retries

  This option specifies how many times to retry starting or resuming the download of the main video or audio file if the connection drops or the server returns a transient error. If set to 0 (infinite), the program will keep retrying indefinitely until it successfully completes the download.

  It handles the downloading of an entire file or a main stream over standard connections (e.g., individual MP4/MKV videos).

  The default value is 10, which is the same as the default value for yt-dlp. 0 means “infinite”

- Fragment Retries

  Many modern websites (such as YouTube) do not stream video as a single large file; instead, they divide it into hundreds of small “fragments” or segments (typically using protocols like HLS or DASH).

  This option determines how many attempts should be made for each individual fragment before giving up. If the network is unstable and a fragment fails to download, the system will retry downloading that specific fragment for the specified number of times before moving on to the next one or aborting the process.

  It manages the downloading of individual video fragments within the context of segmented streaming (DASH, HLS/m3u8, ISM); in other cases, it is simply ignored.

  The default value is 10, which is the same as the default value for yt-dlp. 0 means “infinite”

Geo Restriction

Geo-restriction, limits or completely prevents access to online content based on a user's geographical location. You can setup four way to bypassing geographic restriction:

- **Geo verification proxy**

  Enter the URL of the proxy to use for IP address verification on Geo-restricted sites.

- **Geo bypass**

  Bypass geographic restriction via faking X-Forwarded-For HTTP header. One of "default" (only when known to be useful), or "never" .

- **Geo bypass country**

  Two-letter ISO 3166-2 country code that will be used for explicit geographic restriction bypassing via faking X-Forwarded-For HTTP header.

- **Geo bypass IP block**

  IP range in CIDR (Classless Inter-Domain Routing) notation that will be used similarly to Geo bypass country. CIDR notation is a compact representation of an IP address and its associated network mask.

Authentication

Authentication is related to the site you want to log in to at the time of download action. Please note that some websites do not directly support credential authentication. In this case you could use cookies as a workaround.

Cookies

You might want to take a look at why it’s worth considering these options:

https://github.com/yt-dlp/yt-dlp/wiki/FAQ#how-do-i-pass-cookies-to-yt-dlp

In this tab you will be able to provide a "Netscape HTTP Cookie File" for the purpose of accessing sites that require authentication. This allows yt-dlp to read the active session and avoid blocks such as CAPTCHAs, age restrictions, or login requirements (e.g., videos for subscribers).

Menage the cookie file manually

If you choose this option, you will first need to generate a **Netscape HTTP-formatted cookie file** (with a .txt extension, strictly named "cookies.txt") and then import it into Vidtuber using the "Browse" button.

While there are various ways to generate this file, you can do so directly using yt-dlp: Open your terminal or console and run yt-dlp with the following parameters, for example:

```
yt-dlp --cookies-from-browser firefox --cookies "/home/Paolo/cookies.txt" --no-download
```

Naturally, replace the browser name with your own, and adjust the file paths according to your operating system and actual directory structure.

Once the file has been created, import it using the "Browse" button; its file path will then appear in the adjacent text field.

Extract cookies automatically from your browser

If you are using your preferred web browser to view video content for which you have saved your preferences and data, select this feature and let the yt-dlp extract the cookie files for you. Currently supported browsers are:

Brave, Chrome, Chromium, Edge, Firefox, Opera, Safari, Vivaldi, Whale.

Guide to additional parameters for browser cookies

To extend the “Extract cookies automatically from your browser” functionality, Vidtuber has also implemented a text input field that allows for the addition of optional parameters - specifically the keyring, profile, and container - corresponding to the web browser from which the cookies are to be extracted. These three key parameters can be used either together or individually; in all cases, they must be preceded by their respective prefixes (+), (:), (::), example:

+KEYRING:PROFILE::CONTAINER

If used together, they must follow this specific order and must be separated by the respective prefixes that identify these keys (+), (:), (::).

Here is the specific purpose of each additional parameter:

- +KEYRING (Keyring): Tells `yt-dlp` which system password/key manager to use for decrypting the cookies. This is primarily used on Linux and macOS (e.g., gnomekeyring, kwallet, keychain).
- :PROFILE (Profile): Specifies the browser profile folder or name from which to retrieve cookies (e.g., `chrome:Default` or `firefox:work-profile`). This is useful if you have multiple accounts registered within the same browser.
- ::CONTAINER (Container): A feature specific to Mozilla Firefox. It allows for the extraction of cookies from a particular “Multi-Account Container” - useful if you separate your browsing activities into categories such as Personal, Work, or Banking.

Examples:

+gnomekeyring:~/my/profile/folder::Work

:MyDataProfile

::MyWorkContainer

When you might need to use these additional parameters

When using browser cookies alone is no longer sufficient, or the download fails or `yt-dlp` cannot locate your login credentials. You can provide more specific instructions on where to look for cookies by filling the appropriate text fields: